

***C++ 中的 C**

目录

- 变量的定义
- 常量的定义
- 类型转换
 - static cast & dynamic_cast
 - const cast
 - reinterpret cast

变量的定义

- C: at the beginning of a scope. jump to insert new variable definition;
- C++: anywhere in a scope;
- C++: inside the for, while, if and switch statement. Especially useful in the for statement.

变量定义示例

```
#include <iostream>
using namespace std;

int main() {
    {
        int q = 0;
        for (int i = 0; i < 100; i++) {
            q++;
            int p = 12;
        }
        int p = 1; /* a different p */
    }
    switch(int i = cin.get()){
        case 'A': cout << i << endl; break;
        case 'B': cout << i << endl; break;
        case 'C': cout << i << endl; break;
        default: cout << i << endl;
    }
}
```

常量的定义

- constants in C:
 - An example: `#define ENTER 13`
 - No type checking. `#define PI 3.1415926`
 - No address. `&PI` is invalid.
- constants in C++: a variable but its value can not be changed.
 - An example: `const int x=10`
 - must be initialized.

类型转换

- C 中的类型转换:强制类型转换
 - `float a=1.2; int b=a;`
 - `float a=1.2; int b=(int)a;`
- C++ 中的类型转换:显式类型转换
 - `static_cast` & `dynamic_cast`
 - `const_cast`
 - `reinterpret_cast`

static_cast & dynamic_cast

- castless conversion.

```
int i; float f; f = static_cast<float>(i);
```

- narrowing conversion:

```
int i; float f; i = static_cast<int>(f);
```

- conversion from *void:

```
void * vp; float *fp; fp = static_cast<float*>
```

- static cast与dynamic_cast 的区别:
 - 都可以完成指针或者引用的类型转换。
 - static_cast 要求使用者能确定参与转换的数据类型，将不执行运行时的类型检查，通常更快，但是安全性更差，可能会出现内存越界。
 - dynamic_cast 运行时的类型检查，对不明确的指针转换将失败。尽管dynamic_cast 转换更加安全，但是其只适用于指针或引用，而且运行时类型检查也增加了开销。

const_cast

```
#include <iostream>
using namespace std;

int main() {
    const int i = 10; // 声明一个常量i
    int* j = const_cast<int*>(&i); // 使用const_cast去除const修饰符

    *j = 20; // 尝试修改常量i的值
    cout << i << endl;

    return 0;
}
```

reinterpret_cast

reinterpret_cast 不改变指针和内存位置，但是将重新解释当前指针指向的内存区域的读取方式。

```
struct StudentInfo {  
    char name[20];  
    int age;  
    int gender;  
    float score;  
} std1;  
  
strcpy(std1.name, "zhang san");  
std1.age = 21;  
std1.gender = 0;  
std1.score = 90.5;  
  
char* p = reinterpret_cast<char*>(&std1);
```